

Efficient and Universal Watermarking for LLM-Generated Code Detection

Boquan Li, Zirui Fu, Mengdi Zhang, Peixin Zhang, Jun Sun and Xingmei Wang

Abstract—Large language models (LLMs) have significantly enhanced the usability of AI-generated code, providing effective assistance to programmers. This advancement also raises ethical and legal concerns, such as academic dishonesty and the generation of malicious code. For accountability, it is imperative to detect whether a piece of code is AI-generated. Watermarking is broadly considered a promising solution and has been successfully applied to identify LLM-generated text. However, existing efforts on code are far from ideal, suffering from limited universality and excessive time and memory consumption. In this work, we propose a plug-and-play watermarking approach for AI-generated code detection, named ACW (AI Code Watermarking). ACW is training-free and works by selectively applying a set of carefully-designed, semantic-preserving and idempotent code transformations to LLM code outputs. The presence or absence of the transformations serves as implicit watermarks, enabling the detection of AI-generated code. Our experimental results show that ACW effectively and efficiently detects AI-generated code, preserves code utility, and is resilient against potential code disruptions. Especially, ACW is universal across different LLMs, addressing the limitations of existing approaches.

Index Terms—Large language model, code generation, watermarking, code transformation.

I. INTRODUCTION

LARGE language models (LLMs) [1] mark a milestone in the progress of artificial intelligence (AI) and prompt a variety of applications [2], one of which is code generation. Multiple LLMs, such as ChatGPT [3] and Qwen2.5-Coder [4], are widely adopted for code generation. While LLMs improve programmers' productivity significantly, they can be misused as well. For instance, Langton [5] showed how an attacker could use ChatGPT to construct ransomware. In addition, students may dishonestly leverage LLMs to generate programming assignments. For accountability, it is an urgent need to detect whether a piece of code is AI-generated.

Existing efforts on detecting AI-generated content mostly focus on text [6] and are broadly categorized into two groups. The first group includes passive detectors [7, 8, 9, 10] which build binary classifiers to distinguish between AI-generated and human-written text. The second group focuses on watermarking [11, 12, 13, 14, 15, 16, 17], which actively embeds hidden 'patterns' (a.k.a. watermarks) into AI-generated text, and determines whether given text is generated based on the presence of the pattern. As the state-of-the-art approaches,

Kirchenbauer et al. [11] propose WLLM (Watermarking LLMs), which splits a vocabulary into 'green' and 'red' tokens and softly promotes the 'green' ones during text generation, thereby a given piece of text can be concluded AI-generated if abundant 'green' tokens are present. Giboulot and Furon [12] propose WaterMax, which embeds watermarks by generating multiple candidate text chunks from an LLM and selecting the candidates that maximize a predefined test statistic, and then detects the watermark by evaluating the same statistic on given text.

In comparison to text, detecting AI-generated code based on watermarking remains an emerging task, with state-of-the-art approaches extending from WLLM. Lee et al. [18] propose a selective watermarking technique via entropy thresholding named SWEET, which prompts 'green' tokens only at positions with high entropy, so as to preserve the utility of AI-generated code. Further, Kim et al. [19] propose a syntax token preserving watermarking approach named STONE, which extends SWEET and further improves the utility of code. In addition, some other code watermarking efforts [20, 21, 22, 23, 24] focusing on diverse tasks such as protecting LLM APIs, are not suitable for the task of AI-generated code detection.

However, existing efforts are far from ideal. First, the state-of-the-art approaches demonstrate limited accuracy in detecting LLM-generated code (as our empirical evidence in Section IV). Second, the LLM manipulation-based watermarking strategies present significant time and memory consumption, which slows down LLM code generation. Third, they are restricted by open-source LLMs, lacking universality, and thus cannot adapt to LLM updates and optimizations.

In response to the above problems, in this work, we propose a plug-and-play code watermarking approach for AI-generated code detection, named **ACW** (AI Code Watermarking). The key ingredient of **ACW** is an (extensible) set of 46 carefully-designed, semantic-preserving and idempotent code transformations, which are categorized into refactoring, reordering, and formatting groups, and some of them are designed with certain random factors, such as transformation strategies based on the hashcode of operands, to confuse potential adversaries.

For watermark embedding, given an LLM, **ACW** systematically post-processes the generated code by selectively applying our designed transformations. For watermark identification, given any code, **ACW** identifies the existence of watermarks by checking whether the transformations have been applied. Note that our transformations are designed to be idempotent, enabling the application status of a certain transformation to be easily identified by checking whether applying the transformation repeatedly changes the code or not. Ultimately, the watermarks serve as indicators for AI-generated code detection.

Boquan Li, Zirui Fu, Xingmei Wang are with College of Computer Science and Technology, Harbin Engineering University. Boquan Li, Mengdi Zhang, Peixin Zhang, Jun Sun are with School of Computing and Information Systems, Singapore Management University.

Corresponding to Peixin Zhang at pxzhang@smu.edu.sg and Xingmei Wang at xingmei.wang@hrbeu.edu.cn.

In this work, we mainly make the following contributions.

- We propose **ACW**, a novel code watermarking approach for AI-generated code detection, which is plug-and-play and universal across different LLMs. Moreover, **ACW** is efficient and works in a training-free manner, without the need to access or modify LLMs.
- The semantic-preserving transformations in **ACW** preserve the utility of code after watermarking. Moreover, **ACW** is resilient against potential code disruptions. Even if the embedded watermark is partially modified, the remaining transformations still ensure its discriminability.
- We systematically evaluate **ACW** based on Python code and projects generated using multiple LLMs and prompt datasets. Experimental results demonstrate that **ACW** is effective as well as efficient in detecting AI-generated code, achieving accuracy over 97%, true positive rates over 94%, false positive rates below 1%, and watermarking time below 0.1 seconds average on one piece of code, significantly outperforming existing approaches. Moreover, **ACW** preserves code utility, remains resilient against code disruptions, and is universal across different LLMs.

The rest of this paper is organized as follows. We define our research problem in Section II and detail our approach of **ACW** in Section III. Our experiments are presented in Section IV and a further ablation study is presented in Section V. We then review related work in Section VI and conclude this work in Section VII. Finally, we release our code, data and an online appendix in Section VIII.

II. PRELIMINARY

In this section, we define our research problem and analyze why existing approaches fail to solve the problem satisfactorily.

A. Problem Definition

There are multiple reasons we would like to know whether a piece of code is AI-generated, such as verifying the originality of code submissions from students or employees, or clarifying copyright ownership between humans and AI. The high-level research problem we aim to address is: *Given a piece of code, how can we determine if it is generated by AI?*

In this work, we address the problem based on watermarking, which is broadly considered an effective solution. It works by embedding distinct patterns into AI-generated code, allowing us to conclude whether given code is generated. Formally, we define a code watermarking approach as follows.

Definition 1: A code watermarking approach consists of a function pair (E, I) , where E is a watermark embedding function and I is a watermark identification function. Function E takes a code snippet (or project) C and a watermark ω as inputs, and produces a watermarked code counterpart, denoted as C^ω . Function I takes given code as input and examines the presence of watermarks.

Requirements. We consider a code watermarking approach to address the defined problem satisfactorily, only if it satisfies the following requirements.

- **Discriminability:** $I(C^\omega) \neq I(\mathcal{H})$, where $\mathcal{H}$ represents a human-written code snippet (or project). This ensures the

AI-generated code must be discriminative from human-written code.

- **Efficiency:** $T(G(C) + E(C)) \approx T(G(C))$, where T is a time counting function and G is a code generation function. This ensures the time for watermarking must be negligible, making little impact on the efficiency of code generation.
- **Utility:** $C = C^\omega$ for any code snippet (or project) C and watermark ω , where $=$ denotes functional equivalence. This ensures the watermarked code must maintain the same functionality as the original code before watermarking.
- **Resilience:** $I(\mathcal{A}(C^\omega)) = I(C^\omega)$, where $\mathcal{A}$ represents code optimizations or modifications. This ensures the detectability of the watermark under potential code disruptions.

B. Existing Approaches Fall Short

In the following, we briefly review existing approaches on watermarking LLM-generated code.

Among existing LLM watermarking approaches, most efforts focus on AI-generated text detection [11, 12, 13, 14, 15, 16, 17]. As the state-of-the-art approaches, Kirchenbauer et al. [11] firstly propose WLLM (Watermarking LLMs), which works by splitting a vocabulary into ‘green’ and ‘red’ tokens and softly promoting ‘green’ ones during text generation. As the split of tokens is unknown to adversaries, WLLM then detects whether given text is AI-generated by identifying the presence of the ‘green’ tokens. Giboulot and Furon [12] propose WaterMax, which embeds watermarks by generating multiple candidate text chunks from an LLM and selecting the one with particular distributions that maximize a predefined test statistic. The watermark is then identified by evaluating whether the p -value of the statistic on given text is under a controlled false alarms.

As an emerging task, only limited watermarking approaches focus on AI-generated code detection. Lee et al. [18] propose a selective watermarking technique via entropy thresholding called SWEET, which extends WLLM by prompting ‘green’ tokens at high-entropy positions to preserve the utility of code. Kim et al. [19] further extend SWEET and propose a syntax token preserving watermarking approach called STONE. For watermarking, STONE excludes the tokens critical to language construction, so as to reduce functional degradations.

However, existing state-of-the-art approaches are far from practical, suffering from the following limitations.

Discriminability limitation. Based on our empirical evaluations (as in Section IV-B), these approaches demonstrate limited accuracy in AI-generated code detection. In particular, WLLM, SWEET and STONE show limited ability to search the ‘green’ tokens for watermarking, as code vocabularies are scarcer and the fixed tokens provide limited watermarking space. Similarly, WaterMax fails to produce particularly distributed watermarks in common code in contrast to text.

Efficiency limitation. Based on our empirical evaluations (as in Section IV-C), these approaches show significant time and memory consumption. In particular, the LLM modification-based watermarking approaches (WLLM, SWEET and STONE) interfere with the optimization strategies of LLMs, resulting in additional overheads that slow down code generation. WaterMax forces LLMs to perform extensive repeated generations, leading to time and memory consumption.

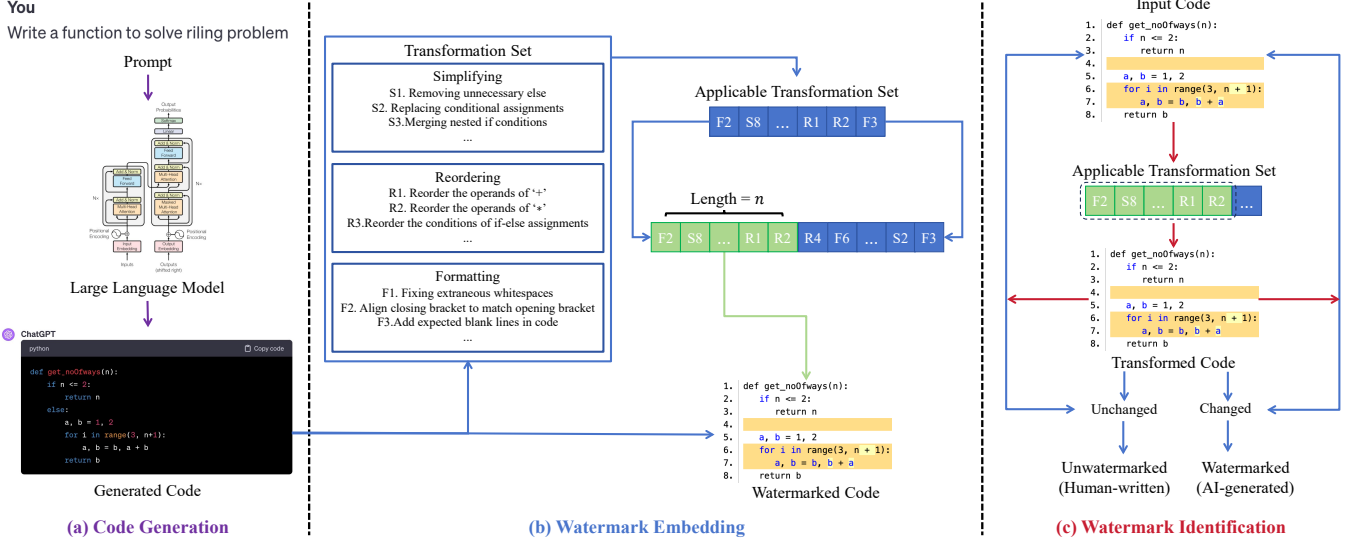


Fig. 1. An overview of ACW.

Universality limitation. Existing white-box watermarking approaches require LLM-specific updates or optimizations that access LLM internals when adapting to different LLMs, which makes them inapplicable to closed-source LLMs and shows limited universality.

Therefore, in this work, we aim to devise effective watermarking strategies that address existing limitations as well as satisfy our requirements.

III. OUR APPROACH

In this section, we present **ACW** in detail, as overviewed in Figure 1. The key idea of **ACW** is to selectively apply a set of carefully-designed code transformations for watermark embedding, and identify watermarks by checking the presence of our transformations. If the watermark is identified to exist in a piece of code, it will be determined as AI-generated.

In the following, we first explain how the embedding function and the identification function work, and then present how the transformations are designed.

A. Watermark Embedding

Given an AI-generated code snippet C , and a set of code transformations $\mathcal{T}$, the embedding function E works as follows.

Firstly, the embedding function systematically analyzes C and identifies the code transformations in $\mathcal{T}$ that are applicable to C . In order to efficiently determine whether a code transformation is applicable or not, the transformations in $\mathcal{T}$ must be designed such that their applicability can be determined statically (based on the abstract syntax trees (AST) [25] in this work), rather than based on certain runtime information. For example, one of our transformations in $\mathcal{T}$ is applicable, as long as there is an `if-then-else` statement in C , i.e., its applicability is determined based on the language construct, rather than determining whether a certain branch of the `if-then-else` statement is infeasible or not.

Algorithm 1 Watermark Embedding

```

1: Input:  $C$  - code snippet
2:    $\mathcal{T}$  - a set of transformations
3:    $\omega$  - the watermark
4:    $n$  - hyperparameters
5: Output:  $C^\omega$  - watermarked code snippet
6:  $T \leftarrow \emptyset$ 
7:  $C^\omega \leftarrow C$ 
8: for  $t \in \mathcal{T}$  do
9:   if  $t$  is applicable to  $C$  then
10:     $T.add(t)$ 
11:   end if
12: end for
13:  $T \leftarrow \text{SORT}(T)$ 
14: for  $t \in T[1:n]$  do
15:    $C^\omega \leftarrow t(C^\omega)$ 
16: end for
17: return  $C^\omega$ 

```

Afterwards, let the set of applicable transformations be T , the embedding function sorts them based on a fixed order (e.g., the alphabetical order of the hashcode of transformations in this work). We reasonably assume that this order is unknown to adversaries and the ordering function is secret. The ordering simplifies the design of code transformations, since different transformations may affect each other; applying the same set of transformations in different orders may result in different code, and adopting a fixed order for applicable transformations potentially ensures fixed results.

Lastly, the embedding function selectively applies n applicable transformations in T to AI-generated code, and checking whether these transformations have been applied or not allows us to tell whether the code is watermarked.

Algorithm 1 details our watermarking embedding function E , with inputs including a code snippet C , a set of transformations

$\mathcal{T}$, a watermark ω and a hyperparameter n . It initializes an empty set of applicable transformations T and a variable C^ω to store the watermarked code at lines 6 and 7. From lines 8 to 12, it iterates through all possible transformations, adding those applicable ones (i.e., applicable to C) to the set T . Next, it sorts T at line 13 and applies the selected n transformations to C^ω from lines 14 to 16. Finally, it returns the watermarked code C^ω at line 17.

B. Watermark Identification

Given a code snippet $\mathcal{C}$ which may or may not be watermarked, and a set of code transformations $\mathcal{T}$, the identification function I aims to identify whether $\mathcal{C}$ is watermarked, or report there is no watermark (i.e., the code is not the product of the embedding function in **ACW**).

Firstly, the identification function I analyzes $\mathcal{C}$ to identify those transformations in $\mathcal{T}$ that are applicable to $\mathcal{C}$, denoted as T . We assume that the set of applicable transformations T identified by I is the same as those identified by the embedding function E . This is only the case if each transformation in T is designed to be applicable if and only if a certain language construct is present in $\mathcal{C}$ and the language construct is not removed after the transformation.

Afterwards, based on the selected n applicable transformations, the identification function I determines whether there are watermarks by checking whether the transformations have been applied on $\mathcal{C}$ already. In this work, the adopted transformations must be idempotent, i.e., $t(C) = t(t(C))$ for any code C and t in $\mathcal{T}$. It means that applying the transformation once or multiple times results in the same code. With this assumption, the identification function simply applies each of the n applicable transformations and checks whether the code is modified or not after the transformation. If the code remains unchanged, it is determined that the transformation has been applied previously. Otherwise, it is determined not to be applied.

We remark a judgment on whether a certain transformation has been applied may be wrong, i.e., $\mathcal{C}$ was naturally written in a way as if the code were the result of the transformation. For example, in the case of a transformation that reorders two operands in an addition operation according to certain orders, the two operands may happen to have the ‘right’ order, as if the transformation has been applied. Such a misjudgment is however unlikely if we set n to be sufficiently large. Let t be an arbitrary transformation in T , and $Pr_{t,C}$ be the probability of a code snippet C naturally satisfying $t(C) = C$. The probability of all n transformations in T denoted as $t_1, t_2, \dots, t_n$, naturally appear to be ‘applied’ is thus $Pr_{t_1,C} * Pr_{t_2,C} * \dots * Pr_{t_n,C}$, which is quite small if n is sufficiently large. For instance, assuming that $Pr_{t,C}$ is 50% for any transformation t , $n = 6$ would make the probability less than 1.6%. Thus, **ACW** is confident enough to identify the existence of watermarks.

Algorithm 2 details our watermark identification function I , with inputs including a code snippet C , a set of pre-defined transformations $\mathcal{T}$ and the hyperparameter n in the watermark embedding function. The output is **Unwatermarked** if no watermark is detected, otherwise, the output is **Watermarked**. From lines 5 to 11, corresponding to the watermark embedding

Algorithm 2 Watermark Identification

```

1: Input:  $C$  - code snippet
2:    $\mathcal{T}$  - a set of transformations
3:    $n$  - hyperparameters
4: Output: Watermarked or Unwatermarked
5:  $T \leftarrow \emptyset$ 
6: for  $t \in \mathcal{T}$  do
7:   if  $t$  is applicable to  $C$  then
8:      $T.add(t)$ 
9:   end if
10: end for
11:  $T \leftarrow \text{SORT}(T)$ 
12: for  $t \in T[1:n]$  do
13:   if  $t(C) \neq C$  then
14:     return Unwatermarked
15:   else
16:      $C \leftarrow t(C)$ 
17:   end if
18: end for
19: return Watermarked

```

algorithm, it starts with initializing an empty set of applicable transformations T and then traverses $\mathcal{T}$ to identify and sort the applicable transformations. From lines 12 to 18, it iterates through the selected n applicable transformations to check the existence of watermarks. If the code is different before and after transformation t , it returns **Unwatermarked**. Otherwise, it updates the code snippet with transformation t . If the code is identical to the input after applying all transformations, it returns **Watermarked**.

C. Transformation

Our designed transformation set is a key component in **ACW**. In the following, we present how we design the transformations.

ACW currently supports 46 transformations categorized into three groups, including refactoring, reordering, and formatting rules. Without loss of generality, we focus on watermarking AI-generated Python code in this work, and it is straightforward that **ACW** can be applied to other programming languages, as our semantic-preserving transformation set can be naturally expanded based on rich language features provided by modern programming languages.

Table I presents an example transformation set in **ACW**, and the remaining transformations are presented in an online appendix (as in Section VIII). We remark that many of the transformations are inspired by existing compilers or code optimizers [26, 27], and some of them are designed with certain ‘random’ factors, such that it is challenging for an adversary to infer what transformations are supported and how they work.

Refactoring transformations are motivated based on the fact that different programming syntax rules can often express the same semantics. For example, a **for** loop can be transformed into a list expression that retains the same semantics. In addition to intuitive rules, this category also includes sophisticated transformations that leverage language features, such as function overloading. For example, for a class with a constructor that

TABLE I
EXAMPLES OF REFACTORING (R), REORDERING (D) AND FORMATTING (F) TRANSFORMATIONS

Number	Rule Description	Original Code Example	Transformed Code Example
R1	Removing unnecessary else.	<pre>def f(a=None): if a is None: return 1 else: # some code here</pre>	<pre>def f(a=None): if a is None: return 1 # some code here</pre>
R2	Converting for loops into list comprehensions.	<pre>cubes = [] for i in range(20): cubes.append(i**3)</pre>	<pre>cubes = [i**3 for i in range(20)]</pre>
R3	Removing unnecessary casts to int, str, float and bool.	<pre>num_hats = 1 update_hat_count(int(num_hats))</pre>	<pre>num_hats = 1 update_hat_count(num_hats)</pre>
R4	Merging multiple comparisons into a single one.	<pre>if pay == "USD" or pay == "EUR": process(pay)</pre>	<pre>if pay in ["USD", "EUR"]: process(pay)</pre>
R5	Inlining a variable to a return in the case when the variable being declared is immediately returned.	<pre>def add_and_return(x, y): result = x + y return result</pre>	<pre>def add_and_return(x, y): return x + y</pre>
R6	Removing unnecessary slice indices.	<pre>numbers[0 : len(numbers)]</pre>	<pre>numbers[:]</pre>
R7	Computing the hash value of a certain class that contains more than one parameter. If the hash value is an odd number, augmenting an unexecuted constructor, otherwise, remaining unchanged.	<pre>class A: def __init__(self, x, y, z): # Given Hash ("A") mod 2 = 1</pre>	<pre>class A: def __init__(self, x, y, z): def __init__(slef, x): #New Constructor</pre>
D1	Comparing the hash values of two operands involved in an '+' operation. Reordering them or remaining unchanged, ensuring operands with larger values always come before the smaller ones.	<pre>for i in range(3, n + 1): a, b = b, a + b return b # Given Hash ("a") < Hash ("b")</pre>	<pre>for i in range(3, n + 1): a, b = b, b + a return b</pre>
D2	Comparing the hash values of two operands involved in an '*' operation. Reordering them or remaining unchanged, ensuring operands with larger values always come before the smaller ones.	<pre>for i in range(3, n + 1): a, b = b, a * b return b # Given Hash ("a") < Hash ("b")</pre>	<pre>for i in range(3, n + 1): a, b = b, b * a return b</pre>
D3	Comparing the hash values of two operands involved in a '>=' or '<=' operator. Reordering them or remaining unchanged, ensuring operands with larger values always come before the smaller ones.	<pre>if n <= left and n <= right: return True # Given Hash ("n") > Hash ("left") # Given Hash ("n") < Hash ("right")</pre>	<pre>if n <= left and right >= n: return True</pre>
F1	Adding excepted blank lines.	<pre>def example_function1(param1, param2): return param1 + param2 def example_function2(param1, param2): return param1 - param2</pre>	<pre>def example_function1(param1, param2): return param1 + param2 def example_function2(param1, param2): return param1 - param2</pre>
F2	Fixing missing whitespaces around operators or keywords.	<pre>def calculate(a, b, c, d): result=a+b-c*d return result</pre>	<pre>def calculate(a, b, c, d): result = a + b - c * d return result</pre>
F3	Aligning closing brackets to match opening brackets.	<pre>def example_function(param1, param2, param3, param4): return param1 + param2 + param3 + param4</pre>	<pre>def example_function(param1, param2, param3, param4): return param1 + param2 + param3 + param4</pre>

takes multiple parameters, we overload the class constructor and introduce an additional one, which only occurs if the hash value of the class name is odd.

Reordering transformations are motivated based on the fact that many operators exhibit commutativity and associativity, i.e., their operands can be reordered without altering code semantics. For example, a comparison expression `n <= right` remains unchanged if the hash value of `n` is greater than or equal to that of `right`, otherwise, it will be transformed into `right >= n`. It is straightforward to see such a transformation can also be extended to other binary operators, such as arithmetic operators (e.g., addition, multiplication), or logical operators like conjunction and disjunction.

Formatting transformations are motivated based on the fact that much human-written code may follow certain standards or styles, e.g., the PEP8 standard [28], and such phenomena can also exist in AI-generated code. Through unifying code

standards and styles as well as rectifying potential formatting flaws, the produced code adheres to consistent 'standardized' coding patterns that constitute discriminating watermarks.

In summary, our transformations are designed with multiple characteristics. First, it should efficiently and statically determine whether a transformation is applicable or not. This is true since we can check whether a transformation is applicable by checking whether certain syntactic patterns exist in code. Second, the applicability of each transformation should remain unchanged after applying the transformation. This is achieved by preserving the syntactic pattern after transformations and can be evident by examining each transformation. Third, all transformations are designed to be semantic-preserving and idempotent, which can be shown by examining the design of each transformation one by one. Lastly, it should be apparent that extending our transformation library with additional rules is straightforward.

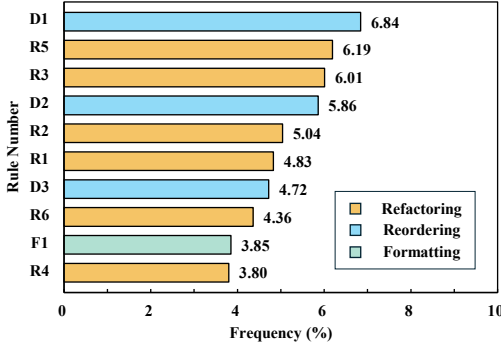


Fig. 2. Statistics of the top-10 contributing transformation rules.

Figure 2 presents statistics of the top-10 applicable transformation rules contributing to code watermarking, where the rule numbering follows that in Table I. As in the figure, the columns in different colors (corresponding to different types of transformations) indicate the relative application frequency of these transformation rules, i.e., the proportion of the frequency each transformation is applied against the sum application frequency across all transformations (110606 times in total), which are counted based on the total of our experimental code in Section IV-A. The top-10 relative application frequency ranges from 3.80% to 6.84%, which reflects the intended balanced application of different transformations in ACW. We will further ablatively explore the effect of our transformation rules from multiple perspectives in Section V.

IV. EXPERIMENT

In this section, we conduct a series of experiments to evaluate the effectiveness of ACW, by answering the following research questions (RQs).

- *RQ1 (Discriminability): Is ACW effective in distinguishing AI-generated code from human-written code?*
- *RQ2 (Efficiency): Is ACW efficient in watermarking AI-generated code?*
- *RQ3 (Utility): Does ACW preserve code utility?*
- *RQ4 (Resilience): How resilient is ACW against potential code disruptions?*

We release our code and data to implement ACW available online, as in Section VIII. All the experiments in this work are conducted on a server with an Intel® Xeon® Gold 6348 CPU (2.60GHz), 240GB of RAM, and an NVIDIA A800 GPU, 80GB of memory.

A. Experiment Setup

ACW is implemented as a plug-and-play tool universal for different LLMs. It currently includes a set of 46 transformations for Python code, and some of which are adopted from code optimization libraries such as *Sourcery* [26] and *Autopep8* [27]. Those transformations containing randomness are implemented using a stable SHA-256 [29] hash algorithm (that is assumed to be replaceable and unknown to adversaries).

Datasets. We collect an extensive set of data encompassing function-level and project-level Python code, which is generated using open-source and closed-source LLMs in combination with

widely adopted prompt benchmarks, as well as multiple-source human-written code, enabling comprehensive evaluations.

We firstly generate a collection of function-level Python code based on the state-of-the-art LLMs including GPT-4 [3], GPT-4o and Qwen2.5-Coder-14B-FP16 [4], with prompt datasets including MBPP [30], APPS [31] and HumanEval+ [32], which are widely adopted in code-related research [18, 19]. Using each LLM, we generate 974, 3035, and 164 pieces of code based on MBPP, APPS and HumanEval+, respectively, producing nine sets of AI-generated code, including MBPP-GPT-4, MBPP-GPT-4o, MBPP-Qwen, APPS-GPT-4, APPS-GPT-4o, APPS-Qwen, HumanEval-GPT-4, HumanEval-GPT-4o and HumanEval-Qwen. We systematically apply ACW to watermark these AI-generated code, and assign each set with the same quantity of human-written code, which is collected from the corresponding benchmark data provided by MBPP, APPS and HumanEval+.

Additionally, we generate a collection of Python projects based on GPT-4, GPT-4o, and a representative LLM agent, ChatDev [33]. ChatDev incorporates a Software Requirement Description Dataset (SRDD), which is the largest project-level prompt dataset to date [34]. Qwen2.5-Coder-14B-FP16 is not adopted as the context processing capability of an open-source LLM cannot support large-scale information flows and multi-round decision-making required for agents. Based on each LLM, we generate 500 projects that functionally decompose 7465 executable-function code, and produce SRDD-GPT-4 and SRDD-GPT-4o testing sets. We systematically apply ACW to watermark these AI-generated projects through watermarking each executable function, and assign each set with the same quantity of human-written code projects from a representative CodeSearchNet [35] challenge dataset.

Baseline Methods. We apply the state-of-the-art AI-generated content detection methods based on watermarking, including WLLM [11], WaterMax [12], SWEET [18] and STONE [19] as our baselines, as detailed in Section II-B. We re-implement the baselines based on the source code provided in [19] (WLLM, SWEET and STONE) and [12] (WaterMax), with hyperparameters identical to the original settings in the literature. Note that we do not conduct any re-training or fine-tuning for fair comparisons. In addition, we evaluate these baselines based on the Qwen LLM, as their white-box watermarking strategies do not apply to those closed-source LLMs such as ChatGPT.

Metrics. We measure the performance of ACW and other baselines in distinguishing AI-generated (a.k.a. positive) or human-written (a.k.a. negative) code, based on the following evaluation metrics.

- $ACC = (TP + TN) / (TP + TN + FP + FN)$ measures the overall performance, where $TP + TN$ refers to the total correctly-identified code and $TP + TN + FP + FN$ refers to the total code.
- $TPR = Recall = TP / (TP + FN)$ measures the performance in identifying watermarks, where TP refers to the correctly-identified AI-generated code and $TP + FN$ refers to the total AI-generated code.
- $FPR = FP / (FP + TN)$ measures false alarms, where FP refers to the incorrectly-identified human-written code and $FP + TN$ refers to the total human-written code.

TABLE II
EVALUATION RESULTS ON DISCRIMINABILITY

Method	Dataset	ACC (%)	TPR (%)	FPR (%)	Precision (%)	F1-Score (%)
ACW	MBPP-GPT-4	98.97	97.94	0	100	98.96
	APPS-GPT-4	98.43	97.03	0.17	99.83	98.41
	HumanEval-GPT-4	97.56	95.12	0	100	97.50
	SRDD-GPT-4	100	100	0	100	100
	MBPP-GPT-4o	98.51	97.02	0	100	98.49
	APPS-GPT-4o	98.14	96.44	0.17	99.82	98.10
	HumanEval-GPT-4o	97.87	95.73	0	100	97.82
	SRDD-GPT-4o	100	100	0	100	100
	MBPP-Qwen	98.81	97.63	0	100	98.80
WLLM	APPS-Qwen	98.15	96.47	0.17	99.82	98.12
	HumanEval-Qwen	97.25	94.51	0	100	97.18
	MBPP-Qwen	68.08	62.98	26.82	70.13	66.36
	APPS-Qwen	68.20	67.32	30.92	68.53	67.92
	HumanEval-Qwen	68.26	70.22	33.70	67.57	68.87
	MBPP-Qwen	57.23	19.29	4.83	79.98	31.08
	APPS-Qwen	59.27	23.35	4.96	82.48	36.40
	HumanEval-Qwen	59.27	22.80	4.27	84.23	35.89
	MBPP-Qwen	75.64	55.91	4.62	92.37	69.66
SWEET	APPS-Qwen	71.20	47.80	5.40	89.85	62.40
	HumanEval-Qwen	74.67	49.95	0.60	98.81	66.36
	MBPP-Qwen	70.84	64.88	23.20	73.66	68.99
STONE	APPS-Qwen	68.36	69.98	33.26	67.78	68.86
	HumanEval-Qwen	76.52	66.46	13.41	83.21	73.90

- $Precision = TP / (TP + FP)$ measures the reliability in avoiding false alarms, where $TP + FP$ refers to the total code that is identified as AI-generated.
- $F1-Score = 2 \times (Precision \times Recall) / (Precision + Recall)$ measures the balanced performance on identifying watermarks and avoiding false alarms.

Note that we determine an AI-generated code project through binomial tests on the watermark identification of its executable functions, with a strict significance threshold at 0.05.

We measure the efficiency of ACW and other baselines in terms of the time (in seconds) and GPU memory (in mebibytes) cost for watermarking. In addition, we measure the utility of AI-generated code before and after watermarking in terms of Pass Rate, i.e., the proportion of test-passing samples among the total. The function-level pass rate is evaluated based on the first-generation code that successfully passes the test cases provided by the corresponding prompt datasets, MBPP, APPS and HumanEval+. The project-level pass rate is evaluated based on the executable functions in code projects that are successfully parsed by syntax trees [36] while ensuring all the syntax tree nodes are valid.

B. RQ1: Discriminability

Through this experiment, we aim to answer: *Is ACW effective in distinguishing AI-generated code from human-written code?*

In the following, we evaluate ACW and other baselines in distinguishing AI-generated code. Table II presents our evaluation results. Based on merged AI-generated and human-written code datasets, we systematically watermark the generated code using ACW and other baselines, and then identify the watermark

to evaluate the discriminability in terms of ACC, TPR, FPR, Precision and F1-Score. For ACW, we set $n = |T|$, i.e., a general setting in which all the applicable transformations are utilized for evaluation. For the baseline methods, we present their optimal results when testing with different thresholds.

```
def lowbit(x):
    return x & -x

# Add expected blank lines
def solve(sum, limit):
    result = []
    for i in range(1, 1 + limit):
        # Reorder the operands of '+' operation
        if lowbit(i) <= sum:
            # Reorder the operands of '<=' operator
            sum -= lowbit(i)
            result.append(i)
        if sum == 0:
            break
    return [-1] if sum != 0 else result
# Merge multiple returns in an if-else statement
```

(a) Success case of ACW

```
from math import gcd

n = input().strip()
s = input().strip()
nums = list(map(int, s))
common_divisor = nums[0]
for num in nums:
    common_divisor = gcd(common_divisor, num)
print("YES" if common_divisor else "NO")
```

(b) Failure case of ACW

Fig. 3. Example cases of ACW, where the transformed (watermarked) elements are highlighted with italics and yellow backgrounds.

As in Table II, **ACW** demonstrates promising discriminability and outperforms other baselines (as the best results in bold), with ACC over 97%, TPR over 94%, FPR below 1%, Precision over 99% and F1-Score over 97% on all datasets. For example, based on MBPP-Qwen, **ACW** achieves ACC of 98.81%, TPR of 97.63%, FPR of 0%, Precision of 100% and F1-Score of 98.80%. The promising results demonstrate the effectiveness of **ACW** in distinguishing AI-generated code from human-written merged sets (in terms of ACC), with balanced performance (in terms of F1-Score) on identifying watermarks (in terms of TPR) and avoiding false alarms (in terms of FPR and Precision). To qualitatively analyze the success and failure cases of **ACW**, as exemplified in Figure 3, the limited failures are caused by some overly simple code that involves almost no logic for semantic-preserving transformations. Note that the overly simple code poses limited authenticity concerns and is commonly considered less interesting for watermarking. Such phenomena do not appear on the project-level SRDD-GPT-4 and SRDD-GPT-4o datasets (with outstanding TPR of 100%), as complex code projects provide more space for watermarking compared to code functions.

Compared with **ACW**, these baselines present unsatisfactory performance. WLLM [11] and WaterMax [12] report promising

results on detecting AI-generated text, but the same is not true for code. For example, based on MBPP-Qwen, WLLM achieves ACC of 68.08%, TPR of 62.98%, FPR of 26.82%, Precision of 70.13%, F1-Score of 66.36%, and WaterMax achieves ACC of 57.23%, TPR of 19.29%, FPR of 4.83%, Precision of 79.98%, F1-Score of 31.08%. SWEET [18] and STONE [19] attempt to improve WLLM through adjusting the token selection strategy for adapting to code, but show limited success. For example, based on MBPP-Qwen, SWEET achieves ACC of 75.64%, TPR of 55.91%, FPR of 4.62%, and STONE achieves ACC of 70.84%, TPR of 64.88%, FPR of 23.20%.

We then qualitatively analyze the watermarking strategies of these baselines (as in Section II-B) based on their success and failure cases. Note that we illustrate the example cases and provide the complete resultant data online (as in Section VIII). As exemplified in Figure 4, WLLM, SWEET and STONE show limited ability to search the ‘green’ tokens for watermarking, as code vocabularies are scarcer and the produced fixed tokens provide limited watermarking space. They even tend to prompt ‘green’ tokens in code comments, which deviates from the goal of code watermarking and lacks practicality in real-world applications. Similarly, as exemplified in Figure 5, WaterMax achieves limited success (with TPR below 25% on all datasets)



Fig. 4. Example cases of WLLM, SWEET and STONE, where the ‘green’ tokens are the candidates modified for watermarking, while the ‘red’ tokens and the excluded ‘gray’ tokens are not involved in the watermarking process.


```
def extract_singly(test_list):
    from collections import Counter

    # Flatten tuples into list
    flat_list = [item for sublist in test_list for item in sublist]

    # Count occurrence of each element
    count_dict = Counter(flat_list)

    # Extract elements that occur once
    result = [item for item, count in count_dict.items() if count == 1]

    return result

-----
def get_Odd_Occurrence(arr, arr_size):
    for i in range(0, arr_size):
        count = 0
        for j in range(0, arr_size):
            if arr[i] == arr[j]:
                count += 1
        if count % 2 != 0:
            return arr[i]
    return -1
```

(a) Success case of WaterMax

(b) Failure case of WaterMax

Fig. 5. Example cases of WaterMax.

in the code with a high density of comments or close to text, but fails to produce particularly distributed watermarks in common code that differs from text. In general, code presents lower flexibility than text and remains challenging for existing LLM watermarking approaches on AI-generated content detection.

Based on this experiment, we conclude that **ACW** is effective in distinguishing AI-generated code from human-written code.

C. RQ2: Efficiency

Through this experiment, we aim to answer: *Is ACW efficient in watermarking AI-generated code?*

In the following, we evaluate **ACW** and other baselines in terms of time (in seconds) and GPU memory (in mebibytes) cost for watermarking, i.e., evaluating whether these watermarking strategies slow down LLM code generation.

Table III presents our evaluation results. In the table, the results outside the brackets indicate the average time and GPU memory on watermarking one piece of code, and the results inside the brackets indicate the corresponding average cost on code generation. Note that the code generation memory required by ChatGPT-related LLMs is not presented, as the closed-source LLMs are running online and the GPU memory is inaccessible. In these baseline methods, the generating and watermarking processes are integrated. For clear observations, we count the watermarking cost based on the difference between the integrated cost and the code generation cost (obtained based on the original LLMs without watermarking strategies applied).

As in Table III, **ACW** demonstrates promising efficiency and outperforms other baselines (as the best results in bold). In terms of watermarking time, **ACW** costs less than 0.1 seconds on all datasets. For example, based on MBPP-Qwen, **ACW** costs 0.03 seconds to watermark one piece of code, which is much less than the code generation time (1.10 seconds). Based on SRDD-GPT-4, it costs an average of 0.83 seconds to watermark one code project, which is also negligible compared to generating a project (160.92 seconds). In terms of GPU memory, **ACW** does not produce any additional overhead. For

TABLE III
EVALUATION RESULTS ON EFFICIENCY

Method	Dataset	Time Cost (s)	GPU Memory (MiB)
ACW	MBPP-GPT-4	0.05 (3.08)	0 (N/A)
	APPS-GPT-4	0.07 (3.28)	0 (N/A)
	HumanEval-GPT-4	0.06 (2.92)	0 (N/A)
	SRDD-GPT-4	0.83 (160.92)	0 (N/A)
	MBPP-GPT-4o	0.02 (2.96)	0 (N/A)
	APPS-GPT-4o	0.05 (3.10)	0 (N/A)
	HumanEval-GPT-4o	0.06 (2.81)	0 (N/A)
	SRDD-GPT-4o	0.79 (127.31)	0 (N/A)
	MBPP-Qwen	0.03 (1.10)	0 (29672)
	APPS-Qwen	0.08 (2.01)	0 (29783)
WLLM	HumanEval-Qwen	0.09 (1.68)	0 (29217)
	MBPP-Qwen	6.83 (1.10)	27611 (29672)
	APPS-Qwen	6.42 (2.01)	28374 (29783)
WaterMax	HumanEval-Qwen	8.94 (1.68)	28032 (29217)
	MBPP-Qwen	9.78 (1.10)	8592 (29672)
	APPS-Qwen	10.42 (2.01)	10280 (29783)
SWEET	HumanEval-Qwen	9.69 (1.68)	8385 (29217)
	MBPP-Qwen	6.03 (1.10)	33917 (29672)
	APPS-Qwen	5.10 (2.01)	42160 (29783)
STONE	HumanEval-Qwen	7.79 (1.68)	32836 (29217)
	MBPP-Qwen	6.54 (1.10)	27589 (29672)
	APPS-Qwen	6.25 (2.01)	28288 (29783)
	HumanEval-Qwen	8.97 (1.68)	27964 (29217)

example, based on MBPP-Qwen, the memory for generating one piece of code remains unchanged (29672 MiB) whether **ACW** is applied or not. The promising results benefit from the post-processing mechanism of **ACW**, where static code transformations require minimal execution time and do not introduce additional GPU overhead for watermarking.

Compared with **ACW**, these baselines present unsatisfactory performance. In terms of time, the watermarking cost of all baselines greatly exceeds the cost for code generation. For example, based on MBPP-Qwen, SWEET costs 6.03 seconds for watermarking, which is much more than the time to generate one piece of code (1.10 seconds) and significantly slows down LLMs. In terms of GPU memory, all the baselines introduce significant overheads. Especially, SWEET requires excessive GPU memory to deal with an LLM with complicated prompts. For example, based on APPS-Qwen, SWEET requires 42160 MiB of memory to watermark one piece of code, which exceeds the memory cost for code generation (29783 MiB). To further analyze the watermarking strategies of these baselines, WLLM, SWEET, and STONE introduce modifications to LLMs that disrupt LLM optimization strategies, leading to a modified LLM with lower efficiency than its original version. WaterMax forces LLMs to perform exponentially repeated generations to produce outputs with particular distributions, resulting in significant time and memory consumption. In contrast, **ACW** does not hinder LLM generations and maintains efficiency, providing practical watermarking solutions in real-world applications.

Based on this experiment, we conclude that **ACW** is efficient in watermarking AI-generated code.

TABLE IV
EVALUATION RESULTS ON UTILITY

Method	Dataset	Pass Rate (%)		Degration (%)
		Original	Watermarked	
ACW	MBPP-GPT-4	71.91	70.21	1.70
	APPS-GPT-4	54.79	54.33	0.46
	HumanEval-GPT-4	89.60	88.40	1.20
	SRDD-GPT-4	100	100	0
	MBPP-GPT-4o	82.13	81.49	0.64
	APPS-GPT-4o	59.93	59.64	0.29
	HumanEval-GPT-4o	83.50	82.80	0.70
	SRDD-GPT-4o	100	100	0
	MBPP-Qwen	63.12	62.29	0.83
	APPS-Qwen	49.52	48.40	1.12
	HumanEval-Qwen	76.80	76.20	0.60
WLLM	MBPP-Qwen	63.12	28.64	34.48
	APPS-Qwen	49.52	17.31	32.21
	HumanEval-Qwen	76.80	27.43	49.37
WaterMax	MBPP-Qwen	63.12	61.90	1.22
	APPS-Qwen	49.52	46.06	3.46
	HumanEval-Qwen	76.80	73.17	3.63
SWEET	MBPP-Qwen	63.12	43.21	19.91
	APPS-Qwen	49.52	20.12	29.40
	HumanEval-Qwen	76.80	43.90	32.90
STONE	MBPP-Qwen	63.12	44.66	18.46
	APPS-Qwen	49.52	22.61	26.91
	HumanEval-Qwen	76.80	51.83	24.97

D. RQ3: Utility

Through this experiment, we aim to answer: *Does ACW preserve code utility?*

In the following, we evaluate the effect of ACW and other baselines on the utility of the AI-generated code before and after watermarking. Table IV presents our results, showing the pass rates of the original and watermarked code sets, and the degradation based on the pass rate difference between the two sets. Note that the pass rates of the original code are not 100% due to the limited code generation capability of LLMs.

As in Table IV, ACW successfully preserves the utility of AI-generated code and outperforms other baselines (as the best results in bold). Especially, the pass rate degradation of ACW is less than 2% on all datasets. For example, based on MBPP-Qwen, the pass rate of the watermarked code is 62.29%, which is degraded by only 0.83% compared to the original pass rate (63.12%). The promising results benefit from our design that all the transformations in ACW are semantic-preserving, and there will inevitably be minor errors resulting from unplanned transformation processes or runtime exceptions on test cases.

Compared with ACW, these baselines show clear impacts on the utility of AI-generated code. For example, based on MBPP-Qwen, SWEET degrades the pass rate from 63.12% to 43.21%, with a degradation of 19.91%. To further analyze the watermarking process, WLLM, SWEET and STONE have promoted LLMs to generate text-driven code that involves syntax errors. For example, in Figure 4 (c), the generated function names `remove_whitespace` and `remove-whitespaces` are inconsistent in the same code snippet. WaterMax preserves

code utility (with degradations below 4%) as it almost fails to produce code watermarks (with TPR below 25% on all datasets, as in Table II) and has no impact on code generation. In general, the optimization of LLMs is sensitive and could be disrupted by LLM modifications, thereby degrading the code generation capability of LLMs.

We additionally evaluate the impact of ACW on large-scale human-built code projects. For standard and fair evaluations, we utilize the project data, test environments and test cases provided by Tests4Py [37], an advanced benchmark for system testing. Based on 507 projects (on average 127582 lines of code per project) corresponding to 507 commit versions collected from 21 Python projects, we systematically apply ACW to them and evaluate the utility before and after watermarking. In particular, we conduct system and unit tests on each project using 100 test cases (i.e., 50700 test cases per test type), where the system test assesses the correctness of project initializations, and the unit test verifies whether the committed issues are resolved. Consequently, 93.31% of the system test cases and 99.20% of the unit test cases are passed on the original projects, and the results remain unchanged after watermarking, further demonstrating the reliability of ACW on preserving code utility.

Based on this experiment, we conclude that ACW preserves the utility of AI-generated code successfully.

E. RQ4: Resilience

Through this experiment, we aim to answer: *How resilient is ACW against potential code disruptions?*

In the following, we evaluate the resilience of ACW by designing four potential attackers who aim to (1) disrupt our watermark through post-optimizations on code, (2) disrupt our watermark through content-focused code modifications, (3) disrupt our watermark through adaptive attacks, and (4) detect the existence of our watermark.

In particular, we implement the disruptive *Attacker 1-3* by applying the corresponding disruptions on both AI-generated (watermarked) and human-written code, and comprehensively evaluate the resilience of ACW from three dimensions, including (1) TPR indicates the discriminability of our watermark with attacks, (2) FPR indicates false alarms that may result from human-written code forgeries, and (3) Pass Rate indicates attack validity, i.e., a valid attack is required to preserve code utility to be consistent with real-world threats. We then implement the *Attacker 4* by measuring the syntactic and functional similarity between the original and watermarked code, i.e., evaluating the imperceptibility of our watermark.

Attacker 1: Code optimizations. The attacker aims to disrupt our watermark through post-optimizations on code, reflecting a real-world threat scenario where code is often optimized by compilers during programming. We implement the attacker using an integrated code transformation toolkit, *Ruff* [38], which integrates a wide range of libraries and transformation rules for Python code. In particular, we adopt two settings in *Ruff* for different levels of attacks, i.e., (1) a default level involving 103 transformations from *Pyflakes* [39] and *Pycodestyle* [40], and (2) a maximum level involving 369 transformations from *Pyflakes*, *Pycodestyle*, *Pylint* [41], *Pyupgrade* [42], *Flake8-bugbear* [43], *Flake8-simplify* [44], *Isort* [45] and *Refurb* [46].

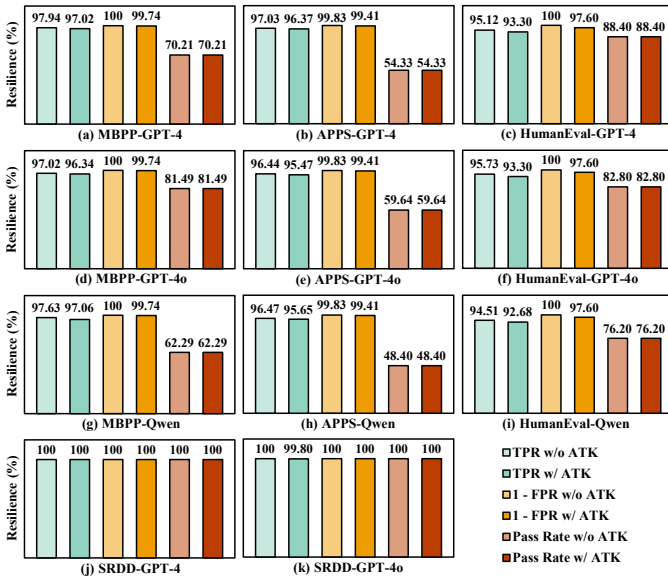


Fig. 6. Resilience results on default-level code optimizations.

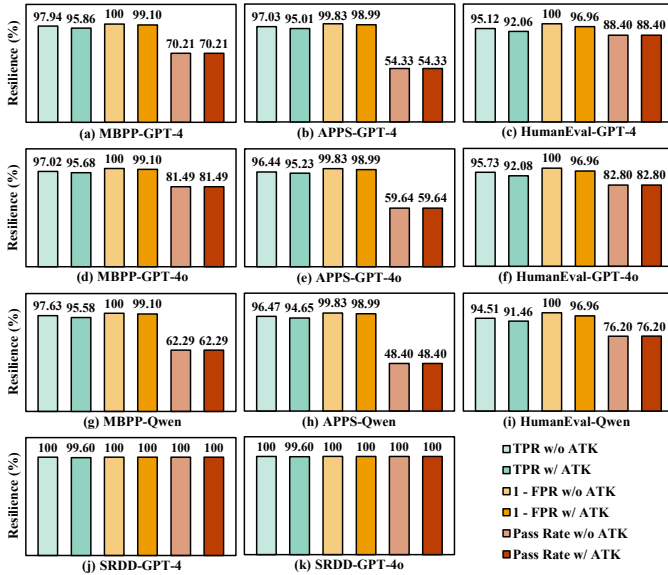


Fig. 7. Resilience results on maximum-level code optimizations.

Figure 6 and Figure 7 present the default-level and maximum-level results on different datasets. In each chart, the blue, orange and red columns respectively present the TPR, FPR and pass rate results with (w/) and without (w/o) attacks (ATK). For clear observations, we present 1–FPR results in the orange columns. As in the charts, **ACW** achieves promising results under different levels of optimizations, maintaining both TPR and FPR on all datasets without pass rate degradations. For example, in Figure 6 (a), **ACW** achieves TPR of 97.02% and 1–FPR of 99.74% on MBPP-GPT-4 under the default-level attack, showing negligible degradations compared to the unattacked TPR of 97.94% and 1–FPR of 100%, and retaining the pass rate of 70.21% unchanged. The promising results demonstrate that our watermarks are discriminative enough to defend against code optimizations (in terms of TPR), as **ACW** contains abundant transformation rules to maintain the

discriminability, even if some transformations are potentially forged to exist in human-written code (in terms of FPR).

Attacker 2: Content-focused code modifications. The attacker aims to disrupt our watermark through content-focused code modifications, following a forceful attack [47, 48] that disrupts watermarks by modifying LLM-output words. To reproduce the attacking strategy on code data, we modify specific content of code involving variable and function names using ChatGPT-4 with the prompt: *Rename the local variable and function names in the following Python code. You must strictly preserve the input argument names and return values, and do not alter the logic, structure, or implementation details of the code.*

Figure 8 presents the resilience results under the modification attack. As in the charts, **ACW** achieves promising TPR and FPR on all datasets without pass rate degradations. For example, in Figure 8 (a), **ACW** achieves TPR of 94.76% and 1–FPR of 99.89% on MBPP-GPT-4, showing negligible degradations compared to the unattacked TPR of 97.94% and 1–FPR of 100%, and retaining the pass rate of 70.21% unchanged. The promising results demonstrate the resilience of **ACW** against content modifications, as the detectability of our watermark is supported by a set of transformation rules, rather than relying on specific content in code.

We additionally explore the resilience of **ACW** in an extreme setting, where an attacker aims to entirely and arbitrarily rewrite the internal logic and control structure of code. Our findings indicate that excessive modifications result in invalid attacks that are beyond real-world threat models, as the utility of code is significantly destroyed by the modifications. In contrast, the above **Attacker 2** provides a valid direction for reproducing the typical content modification attack on code. As the attacker that violates validity prerequisites deviates from our evaluation objectives, we provide the supplementary results in an online appendix (as in Section VIII).

Attacker 3: Adaptive attacks. We assume that our watermark embedding knowledge in **ACW** can be leaked to the attacker, who aims to disrupt our watermark through adaptive attacks. We implement the attacker by fine-tuning a CodeT5+ [49] (770M) LLM based on paired code sequences after and before watermarking. The objective of the attacker LLM is to learn the code-to-code transformation that recovers a piece of watermarked code into its original version.

Figure 9 presents the resilience results under the adaptive attack. It is first observed that the attack is generally valid in terms of pass rate results, e.g., in Figure 9 (a), the pass rate degrades from 70.21% to 63.74% on MBPP-GPT-4 after the attack, indicating that the utility of some code is destroyed, but the degree is within the scope of real-world threat models. We further remark that the adaptive attack is forceful as the attacker LLM has learned the watermark embedding knowledge for recovering our watermarked code. Under such premises, **ACW** is basically resilient against the adaptive attack with certain TPR degradations and stable FPR results, e.g., in Figure 9 (a), the TPR is 71.56% on MBPP-GPT-4 versus the unattacked TPR of 97.94%. The results indicate that despite multiple distinct transformations in code being destroyed by adaptive modifications, the remainders are still resilient to retain the discriminability.

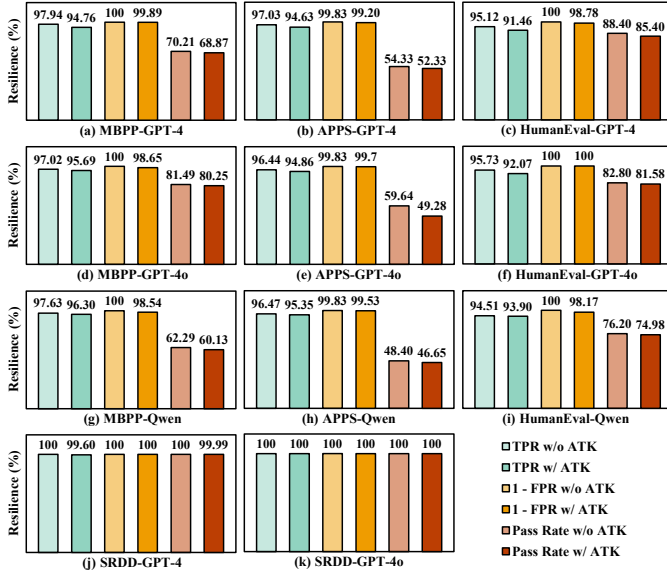


Fig. 8. Resilience results on content-focused code modifications.

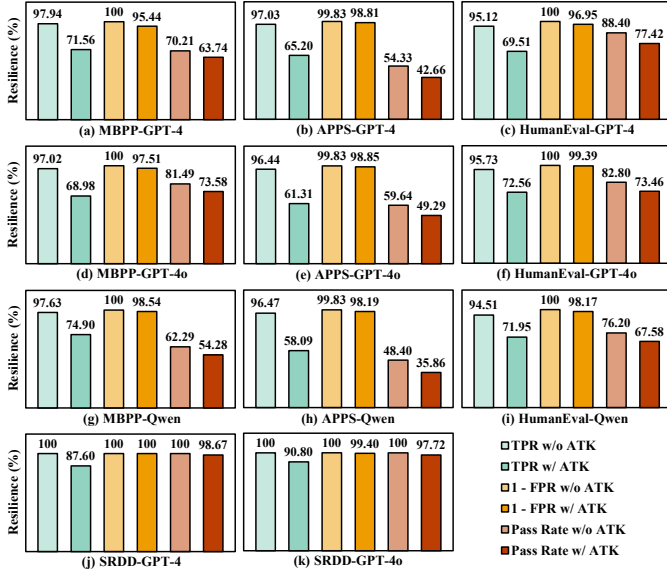


Fig. 9. Resilience results on adaptive attacks.

Attacker 4: Watermark existence detection. The attacker aims to detect the existence of our watermark, thereby taking further disruptive actions as the above *Attacker 1-3*. In particular, we assume an adaptive attacker who is accessible to our watermark embedding knowledge and is able to craft both versions of code before and after watermarking. The attacker then evaluates the invisibility of our watermark based on the functional semantic similarity (in terms of CodeBERTScore [50]) and structural semantic similarity (in terms of CodeBLEU [51]) between the original and watermarked code. Note that CodeBERTScore and CodeBLEU are advanced metrics that measure code similarly comprehensively, providing forceful tools for the attacker to detect the existence of our watermark.

Table V presents the invisibility results containing CodeBERTScore, CodeBLEU and their average scores, where the full score is 100. For standard and fair evaluations, each

TABLE V
EVALUATION RESULTS ON INVISIBILITY

Dataset	CodeBERTScore	CodeBLEU	Average
MBPP-GPT-4	99.10	88.35	93.73
APPS-GPT-4	98.80	90.96	94.88
HumanEval-GPT-4	99.04	89.44	94.24
SRDD-GPT-4	99.32	92.06	95.69
MBPP-GPT-4o	99.05	88.69	93.87
APPS-GPT-4o	99.27	92.99	96.13
HumanEval-GPT-4o	98.95	89.20	94.08
SRDD-GPT-4o	99.24	93.78	96.51
MBPP-Qwen	98.78	89.67	94.23
APPS-Qwen	98.07	89.67	93.87
HumanEval-Qwen	98.81	90.01	94.41

CodeBERTScore and CodeBLEU result is measured as the average score of the total code in a dataset, with higher scores indicating better invisibility of our watermark. As shown in the table, **ACW** achieves promising results with all average scores over 93, e.g., 93.73 on MBPP-GPT-4, indicating high similarity between the original and watermarked code. The invisibility benefits from our semantic-preserving design for the transformation rules in **ACW**, making it challenging for the attacker to detect the existence of our watermark.

Based on this experiment, we conclude that ACW is resilient against potential code disruptions.

V. ABLATION STUDY

In this section, we perform ablation studies to explore the effect of multiple factors on **ACW**, including (1) transformation idempotence, (2) transformation orders, (3) watermark overlaps and (4) code lengths. Note that the following ablative results are counted based on all the function-level watermarked code in Section IV-A, as the project-level results are 100% in different aspects (benefiting from abundant watermarking space), which produces opposite effects for analyzing the ablative results.

A. Transformation Idempotence

We start with evaluating the idempotence of our transformations, i.e., applying a certain transformation once or multiple times results in the same code.

Figure 10 presents our idempotence results. As in the chart, we respectively apply the top-10 contributing transformation rules (as in Figure 2) over five iterations to the total watermarked code (in the horizontal axis), and count the idempotence ratio of each iteration (in the vertical axis), i.e., the proportion of code that remains unchanged under a given transformation among the total code. It is shown by different colors of curves that only scarce watermarked code (less than 0.2%) reveals changes under the first additional transformation, and remains unchanged in the subsequent iterations. The promising results demonstrate that our designed transformations maintain perfect idempotence, ensuring the effectiveness of **ACW**.

B. Transformation Order

In the following, we evaluate whether the application order of transformation rules affects the effectiveness of our watermark.

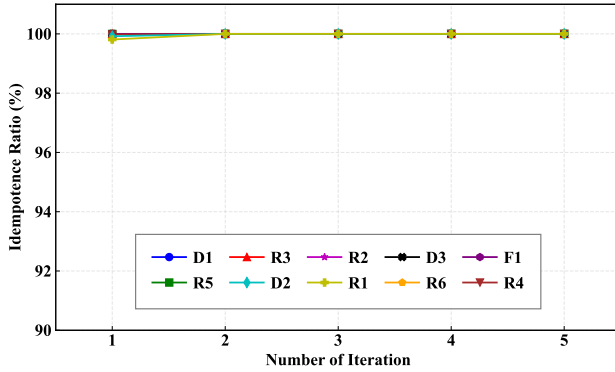


Fig. 10. Ablative results on the idempotence of transformations.

TABLE VI
ABLATIVE RESULTS ON TRANSFORMATION ORDERS

Group	TPR (%)	Pass Rate (%)
Random Seed 1	97.03	59.08
Random Seed 2	96.98	59.10
Random Seed 3	97.10	59.11

TABLE VII
ABLATIVE RESULTS ON OVERLAPPED WATERMARKS

Group	Embedding	Identification	TPR (%)	Fluctuation (%)
1	Set A	Set A	97.36	—
2	Set A→Set B	Set A	96.67	0.69
3	Set A→Set B→Set C	Set A	97.04	0.32
4	Set A→Set B	Set B	97.56	—
5	Set A→Set B→Set C	Set B	97.78	0.22
6	Set A→Set B→Set C	Set C	98.03	—
7	Set D	Set D	97.21	—
8	Set D→Set E	Set D	97.03	0.18
9	Set D→Set E→Set F	Set D	97.19	0.02
10	Set D→Set E	Set E	97.30	—
11	Set D→Set E→Set F	Set E	97.13	0.17
12	Set D→Set E→Set F	Set F	97.82	—
13	Set G	Set G	97.11	—
14	Set G→Set H	Set G	96.74	0.37
15	Set G→Set H→Set I	Set G	96.91	0.02
16	Set G→Set H	Set H	97.31	—
17	Set G→Set H→Set I	Set H	97.79	0.48
18	Set G→Set H→Set I	Set I	97.93	—

Table VI presents our ablative results with different orders of transformations, including the average TPR and pass rates over the total watermarked code. As in the table, three groups of code are watermarked based on different application orders of 46 transformation rules, where each order is generated using a distinct random seed. It is shown by the results that three groups of transformations ordered by different random seeds achieve approximate TPR and pass rates (with differences below 0.2%), demonstrating the stability and reliability of **ACW**.

C. Watermark Overlap

We next evaluate whether applying our watermarks multiple times (with different transformation sets representing different LLMs) affects the effectiveness of **ACW**.

Table VII presents our evaluation results. To take the first six experimental groups as an example, three transformation sets

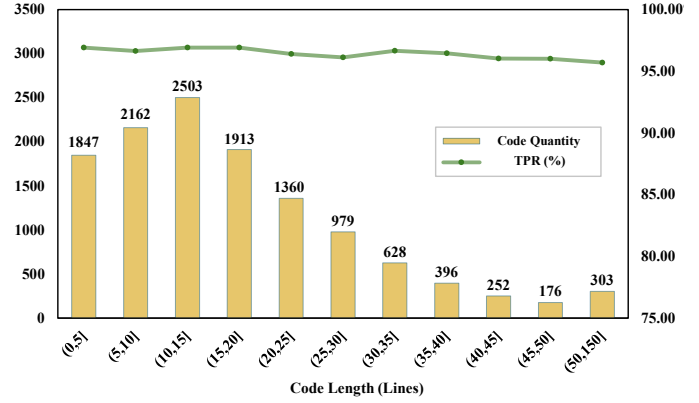


Fig. 11. Ablative results on the effect of code length on discriminability.

(Set A, Set B and Set C), each containing 23 transformations, are selected using the random seed 1. Note that the transformations in each set are randomly intersected rather than fully overlapping, as the latter has been evaluated in Section V-A, where repeated transformations tend to produce the same code (as the idempotence results in Figure 10). In particular, we sequentially watermark our code using the transformations in Set A, Set B and Set C, producing overlapped watermarks. In the table, Group 1 presents the initial TPR results by identifying the watermark in Set A. Group 2-3 presents the TPR results and TPR fluctuations (compared to the initial TPR in Group 1) of Set A under sequential watermarks using Set B and Set C. Similarly, Group 4-5 presents the initial and overlapped results of Set B, and Group 6 presents the final TPR results of Set C. It is shown in the table that the TPR results remain stable (with TPR fluctuations below 1%) when applying our transformations multiple times, which further demonstrates the discriminability and resilience of **ACW**.

D. Code length

In the end, we explore the effect of code lengths on the discriminability of our watermark.

Figure 11 presents our ablative results. As the horizontal axis in the chart, we group our watermarked code by different ranges of code length, where the yellow columns count the quantity of code in each group. As the green curve in the chart, we then identify the watermark in each group and evaluate the discriminability in terms of TPR. It is shown by the green curve that **ACW** presents stable TPR results over 95% across all code lengths, which demonstrates our watermark can be successfully embedded and identified over different code lengths.

VI. RELATED WORK

In this section, we review related work on AI-generated text and code detection, which is broadly related to this work. In particular, existing approaches are commonly categorized into passive and active detectors.

A. Passive Detection

The passive approaches involve building binary classifiers to distinguish AI-generated or human-written content. Currently,

there is little work specifically focusing on AI-generated code, instead, they aim to detect any AI-generated content, regardless of text or code. Mitchell et al. [7] show that AI-generated text tends to occupy the negative curvature region of LLM log probability functions, proposing DetectGPT as a detection tool. Bao et al. [8] introduce a conditional probability curvature to highlight discrepancies between AI and human text, presenting Fast-DetectGPT that improves the efficiency of DetectGPT. Hu et al. [9] report that AI detectors lack robustness against LLM-based paraphrasing and propose RADAR, an adversarial learning-based detection framework. GPTZero [10] attempts to uncover the shortcomings of LLMs and propose a practical web application to detect AI-generated content.

While these passive detectors are effective at detecting AI-generated text, Wang et al. [52] report that existing approaches cannot generalize to code unless extensive additional fine-tuning is performed. Additionally, similar to other domains like image forgery detection [53], passive detectors are considered less generalizable than watermarking approaches, in which the embedded patterns are more stable than general features.

B. Active Detection

The active approaches involve embedding hidden watermarks into AI-generated content and detecting AI-generated or human-written content based on the presence of watermarks.

Similar to passive detectors, most of the LLM watermarking efforts focus on text. As a pioneer approach, Kirchenbauer et al. [11] propose WLLM, which splits a vocabulary into ‘green’ and ‘red’ tokens, prompts ‘green’ tokens during text generation, and identifies the presence of ‘green’ tokens to determine whether certain text is AI-generated. Giboulot and Furon [12] propose WaterMax, which generates multiple text chunks, selects the one that maximizes a predefined test statistic for watermarking, and detects watermarks by evaluating the test statistic. Zhao et al. [13] propose Unigram-Watermark, which extends WLLM with a fixed token grouping strategy. Liu and Bu [14] propose AWTI, which performs adaptive watermark token identification, selectively targeting high-entropy tokens while leaving low-entropy tokens unchanged. Christ et al. [15] propose a cryptographically-inspired undetectable watermark, detectable only with secret keys for preventing detectable changes to output distributions. Zhang et al. [16] propose REMARK-LLM, a watermarking framework comprising an encoding module for embedding binary signatures, a reparameterization module for mapping signatures into sparse tokens, and a decoding module for watermark extraction. Dathathri et al. [17] propose SynthID-Text, an LLM text watermarking scheme that operates by modifying the sampling procedure during text generation.

Compared to text watermarking, detecting AI-generated code based on watermarking remains an emerging task, with limited research to date. Based on the red-green watermarking strategy from WLLM [11], Lee et al. [18] propose a selective watermarking technique via entropy thresholding called SWEET, which extends WLLM by prompting ‘green’ tokens only at high-entropy positions, so as to preserve code utility. Kim et al. [19] further extend SWEET and propose a syntax token-preserving watermarking approach, called STONE, which excludes tokens critical to code execution, thereby avoiding utility degradations.

Based on our experimental evaluations in Section IV, existing state-of-the-art approaches demonstrate limited accuracy in detecting LLM-generated code, and present significant time and memory consumption that slows down LLMs.

In addition to the watermarking approaches, some research explores attacking strategies for benchmarking the resilience or robustness of LLM watermarks, and existing efforts mainly focus on AI-generated text. For example, focusing on the red-green watermarking strategy as in WLLM [11], Zhang et al. [54] propose a ‘green’ list stealing attack, which formalizes a mixed integer programming problem, with the aim to find a minimal available ‘green’ list constrained by a set of rules. Pang et al. [47, 48] explore the trade-off in the robustness, utility and usability of LLM watermarks, and propose a content-based spoofing attack that results in the watermarked text inaccurate through content modifications. Based on our experimental evaluations in Section IV, ACW is resilient against multiple types of typical attacks.

Apart from watermarking approaches for AI-generated code detection, some research explores code watermarking in other applications. Guan et al. [20] extend WLLM and propose CodeIP, which embeds information with crucial provenance details for safeguarding the intellectual property of LLMs. Ning et al. [21] also extend WLLM and propose MCGMark, which selects tokens based on probabilistic outliers to trace malicious code. Li et al. [22] introduce ToSyn, a watermarking scheme that stealthily tweaks the distribution among synonym tokens in LLM outputs, for protecting LLM APIs from remote imitation attacks. Sun et al. [23] propose CodeMark, which embeds user-defined imperceptible watermarks into code datasets to trace their usage in code completion models. Yang et al. [24] introduce SrcMarker, which uses learning-based embedding and extraction strategies to select rules for watermarking, focusing on traditional source code watermarking unrelated to LLMs. These methods explore diverse applications of code watermarking, beyond the task of AI-generated code detection.

VII. CONCLUSION AND DISCUSSION

In this work, we propose a plug-and-play code watermarking approach, i.e., ACW, for AI-generated code detection. ACW watermarks AI-generated code based on a series of carefully-designed, semantic-preserving and idempotent code transformations. ACW detects AI-generated code by identifying the existence of our watermarks, i.e., whether our transformations have been applied. Experimental results demonstrate that ACW is effective in detecting AI-generated code, preserves code utility, and is resilient against code optimizations. Especially, ACW is efficient and universal across different LLMs, addressing the limitations in existing work.

Finally, we discuss potential validity threats regarding ACW. First, ACW currently includes three groups of 46 transformations, which are demonstrated effective in watermarking and identifying AI-generated Python code. It is straightforward that our rule library can be naturally expanded from both aspects of rule numbers and other languages, based on rich language features provided by modern programming languages. Second, ACW focuses on an emerging task driven by the development

of LLMs, i.e., AI-generated code detection. In the future, our watermarking strategies have the potential to extend to broader applications, e.g., tracing the authorship of open-source code or projects, verifying the integrity of code over time, and identifying reused code in copyright-sensitive projects.

VIII. CODE AND DATA AVAILABILITY

We release our code and data resulting from this work, along with an online appendix presenting our complete transformation rules and supplementary experiments, available at <https://github.com/Noelle1831-k/ACW>.

ACKNOWLEDGMENT

We sincerely appreciate the editors and reviewers for their insightful feedback and constructive suggestions throughout the review process, which have contributed to the improvement and refinement of this manuscript.

REFERENCES

- [1] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong *et al.*, “A survey of large language models,” *arXiv preprint arXiv:2303.18223*, 2023.
- [2] Y. Liu, T. Han, S. Ma, J. Zhang, Y. Yang, J. Tian, H. He, A. Li, M. He, Z. Liu *et al.*, “Summary of chatgpt-related research and perspective towards the future of large language models,” *Meta-Radiology*, p. 100017, 2023.
- [3] OpenAI, “Chatgpt,” 2023, accessed: 2025. [Online]. Available: <https://chat.openai.com/>
- [4] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, T. Liu, J. Zhang, B. Yu, K. Lu *et al.*, “Qwen2. 5-coder technical report,” *arXiv preprint arXiv:2409.12186*, 2024.
- [5] A. Langton, “Juniper networks: Using chatgpt to generate native code malware,” 2023, accessed: 2025. [Online]. Available: <https://blogs.juniper.net/en-us/threat-research/using-chatgpt-to-generate-native-code-malware>
- [6] X. Zhao, S. Gunn, M. Christ, J. Fairuze, A. Fabrega, N. Carlini, S. Garg, S. Hong, M. Nasr, F. Tramer *et al.*, “Sok: Watermarking for ai-generated content,” in *IEEE Symposium on Security and Privacy*, 2025, pp. 2621–2639.
- [7] E. Mitchell, Y. Lee, A. Khazatsky, C. D. Manning, and C. Finn, “Detectgpt: zero-shot machine-generated text detection using probability curvature,” in *International Conference on Machine Learning*, 2023, pp. 24 950–24 962.
- [8] G. Bao, Y. Zhao, Z. Teng, L. Yang, and Y. Zhang, “Fast-detectgpt: Efficient zero-shot detection of machine-generated text via conditional probability curvature,” in *International Conference on Learning Representations*, 2024.
- [9] X. Hu, P.-Y. Chen, and T.-Y. Ho, “Radar: robust ai-text detection via adversarial learning,” in *International Conference on Neural Information Processing Systems*, vol. 36, 2023, pp. 15 077–15 095.
- [10] GPTZero, “Gptzero official website,” 2025, accessed: 2025. [Online]. Available: <https://gptzero.me/>
- [11] J. Kirchenbauer, J. Geiping, Y. Wen, J. Katz, I. Miers, and T. Goldstein, “A watermark for large language models,” in *International Conference on Machine Learning*, 2023, pp. 17 061–17 084.
- [12] E. Giboulot and T. Furon, “Watermax: breaking the llm watermark detectability-robustness-quality trade-off,” *International Conference on Neural Information Processing Systems*, vol. 37, pp. 18 848–18 881, 2024.
- [13] X. Zhao, P. V. Ananth, L. Li, and Y.-X. Wang, “Provable robust watermarking for ai-generated text,” in *International Conference on Learning Representations*, 2024.
- [14] Y. Liu and Y. Bu, “Adaptive text watermark for large language models,” in *International Conference on Machine Learning*, 2024, pp. 30 718–30 737.
- [15] M. Christ, S. Gunn, and O. Zamir, “Undetectable watermarks for language models,” in *Annual Conference on Learning Theory*, 2024, pp. 1125–1139.
- [16] R. Zhang, S. S. Hussain, P. Neekhara, and F. Koushanfar, “Remark-llm: A robust and efficient watermarking framework for generative large language models,” in *USENIX Security Symposium*, 2024, pp. 1813–1830.
- [17] S. Dathathri, A. See, S. Ghaisas, P.-S. Huang, R. McAdam, J. Welbl, V. Bachani, A. Kaskasoli, R. Stanforth, T. Matejovicova *et al.*, “Scalable watermarking for identifying large language model outputs,” *Nature*, vol. 634, no. 8035, pp. 818–823, 2024.
- [18] T. Lee, S. Hong, J. Ahn, I. Hong, H. Lee, S. Yun, J. Shin, and G. Kim, “Who wrote this code? watermarking for code generation,” in *Annual Meeting of the Association for Computational Linguistics*, 2024, pp. 4890–4911.
- [19] J. Kim, S. Park, and Y.-S. Han, “Marking code without breaking it: Code watermarking for detecting llm-generated code,” *arXiv preprint arXiv:2502.18851*, 2025.
- [20] B. Guan, Y. Wan, Z. Bi, Z. Wang, H. Zhang, P. Zhou, and L. Sun, “Codeip: A grammar-guided multi-bit watermark for large language models of code,” in *Findings of the Association for Computational Linguistics: EMNLP*, 2024, pp. 9243–9258.
- [21] K. Ning, J. Chen, Q. Zhong, T. Zhang, Y. Wang, W. Li, Y. Zhang, W. Zhang, and Z. Zheng, “Mcgmarm: An encodable and robust online watermark for llm-generated malicious code,” *arXiv preprint arXiv:2408.01354*, 2024.
- [22] Z. Li, C. Wang, S. Wang, and C. Gao, “Protecting intellectual property of large language model-based code generation apis via watermarks,” in *ACM SIGSAC Conference on Computer and Communications Security*, 2023, pp. 2336–2350.
- [23] Z. Sun, X. Du, F. Song, and L. Li, “Codemark: Imperceptible watermarking for code datasets against neural code completion models,” in *Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering*, 2023, pp. 1561–1572.
- [24] B. Yang, W. Li, L. Xiang, and B. Li, “Srcmarker: Dual-channel source code watermarking via scalable code transformations,” in *IEEE Symposium on Security and Privacy*, 2024, pp. 4088–4106.
- [25] PythonAST, “Abstract syntax trees,” 2024, accessed: 2025.

- [Online]. Available: <https://docs.python.org/3/library/ast.html>
- [26] Sourcery, "Sourcery github repository," 2024, accessed: 2025. [Online]. Available: <https://github.com/sourcery-ai/sourcery>
- [27] H. Hattori, "Autopep8 github repository," 2024, accessed: 2025. [Online]. Available: <https://github.com/hhatto/autopep8>
- [28] A. C. Guido van Rossum, Barry Warsaw, "Pep 8 style guide for python code," 2013, accessed: 2025. [Online]. Available: <https://peps.python.org/pep-0008>
- [29] A. W. Appel, "Verification of a cryptographic primitive: Sha-256," *ACM Transactions on Programming Languages and Systems*, vol. 37, no. 2, pp. 1–31, 2015.
- [30] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le *et al.*, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021.
- [31] D. Hendrycks, S. Basart, S. Kadavath, M. Mazeika, A. Arora, E. Guo, C. Burns, S. Puranik, H. He, D. Song *et al.*, "Measuring coding challenge competence with apps," in *Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2021.
- [32] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation," in *International Conference on Neural Information Processing Systems*, vol. 36, 2023, pp. 21 558–21 572.
- [33] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong *et al.*, "Chatdev: Communicative agents for software development," in *Annual Meeting of the Association for Computational Linguistics*, 2024, pp. 15 174–15 186.
- [34] K. Liu, Y. Pan, Y. Xiang, D. He, J. Li, Y. Du, and T. Gao, "Projecteval: A benchmark for programming agents automated evaluation on project-level code generation," in *Findings of the Association for Computational Linguistics*, 2025, pp. 20 205–20 221.
- [35] H. Husain, H.-H. Wu, T. Gazit, M. Allamanis, and M. Brockschmidt, "Codesearchnet challenge: Evaluating the state of semantic code search," *arXiv preprint arXiv:1909.09436*, 2019.
- [36] Tree-Sitter, "Treesitter github repository," 2024, accessed: 2025. [Online]. Available: <https://tree-sitter.github.io/tree-sitter/>
- [37] M. Smytzek, M. Eberlein, B. Serce, L. Grunske, and A. Zeller, "Tests4py: A benchmark for system testing," in *Companion Proceedings of the ACM International Conference on the Foundations of Software Engineering*, 2024, pp. 557–561.
- [38] C. Marsh, "Ruff github repository," 2025, accessed: 2025. [Online]. Available: <https://github.com/astral-sh/ruff>
- [39] Pyflakes, "Pyflakes github repository," 2005, accessed: 2025. [Online]. Available: <https://github.com/PyCQA/pyflakes>
- [40] Pycodestyle, "Pycodestyle github repository," 2012, accessed: 2025. [Online]. Available: <https://github.com/PyCQA/pycodestyle>
- [41] Pylint, "Pylint github repository," 2003, accessed: 2025. [Online]. Available: <https://github.com/pylint-dev/pylint>
- [42] Asottile, "Pyupgrade github repository," 2018, accessed: 2025. [Online]. Available: <https://github.com/asottile/pyupgrade>
- [43] Flake8bugbear, "Flake8-bugbear github repository," 2016, accessed: 2025. [Online]. Available: <https://github.com/PyCQA/flake8-bugbear>
- [44] MartinThoma, "Flake8-simplify github repository," 2021, accessed: 2025. [Online]. Available: <https://github.com/MartinThoma/flake8-simplify>
- [45] Isort, "Isort github repository," 2013, accessed: 2025. [Online]. Available: <https://github.com/PyCQA/isort>
- [46] Dosisd, "Refurb github repository," 2022, accessed: 2025. [Online]. Available: <https://github.com/dosisod/refurb>
- [47] Q. Pang, S. Hu, W. Zheng, and V. Smith, "No free lunch in llm watermarking: Trade-offs in watermarking design choices," *International Conference on Neural Information Processing Systems*, vol. 37, pp. 138 756–138 788, 2024.
- [48] Q. Pang, S. Hu, W. Zheng, and V. S., "Attacking llm watermarks by exploiting their strengths," in *ICLR 2024 Workshop on Secure and Trustworthy Large Language Models*, 2024.
- [49] Y. Wang, H. Le, A. D. Gotmare, N. D. Bui, J. Li, and S. C. Hoi, "Codet5+: Open code large language models for code understanding and generation," in *Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2023, pp. 1069–1088.
- [50] S. Zhou, U. Alon, S. Agarwal, and G. Neubig, "Codebertscore: Evaluating code generation with pretrained models of code," in *Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 13 921–13 937.
- [51] S. Ren, D. Guo, S. Lu, L. Zhou, S. Liu, D. Tang, N. Sundaresan, M. Zhou, A. Blanco, and S. Ma, "Codebleu: a method for automatic evaluation of code synthesis," *arXiv preprint arXiv:2009.10297*, 2020.
- [52] J. Wang, S. Liu, X. Xie, and Y. Li, "An empirical study to evaluate aigc detectors on code content," in *IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 844–856.
- [53] X. Lin, J.-H. Li, S.-L. Wang, F. Cheng, X.-S. Huang *et al.*, "Recent advances in passive digital image security forensics: A brief review," *Engineering*, vol. 4, no. 1, pp. 29–39, 2018.
- [54] Z. Zhang, X. Zhang, Y. Zhang, L. Y. Zhang, C. Chen, S. Hu, A. Gill, and S. Pan, "Stealing watermarks of large language models via mixed integer programming," in *Annual Computer Security Applications Conference*. IEEE, 2024, pp. 46–60.